

Reemplazo de Páginas bajo Distintos Patrones de Acceso: Comparación Experimental de FIFO y LRU frente a la Cota Óptima

Isaac Felipe Morún Moreira

Universidad Invenio — Licenciatura en Tecnologías de Información y Comunicación Empresarial

TIIT2007 – Sistemas Operativos | Agosto 2026

1. Resumen

Este documento presenta el diseño, la implementación y la evaluación experimental de un simulador de reemplazo de páginas desarrollado en C++17, que compara los algoritmos FIFO y LRU sobre una misma cadena de referencias con el número de marcos configurable. Se incorpora además el algoritmo óptimo de Belady, no exigido por el enunciado, cuya función es servir de cota inferior: sin una referencia de ese tipo no es posible determinar si una cifra de fallos concreta deja margen de mejora o ya se encuentra próxima al límite teórico. La corrección de las tres implementaciones se verificó de forma externa reproduciendo los valores publicados en la literatura para la cadena canónica de referencia, con quince, doce y nueve fallos respectivamente sobre tres marcos. El protocolo experimental abarcó cinco cadenas y hasta seis tamaños de marco, con especial atención a tres patrones de acceso contruidos deliberadamente —con localidad, secuencial y aleatorio— para contrastar con datos propios la afirmación de que LRU aventaja a FIFO únicamente cuando la carga exhibe localidad de referencia. Los resultados confirman esa hipótesis en los tres escenarios y añaden dos hallazgos: el barrido secuencial produce una tasa de fallos del cien por ciento en ambos algoritmos incluso con dieciséis marcos, y la anomalía de Belady se reproduce y se detecta automáticamente, manifestándose exclusivamente en FIFO sobre las treinta combinaciones evaluadas.

Palabras clave—memoria virtual, paginación, fallo de página, reemplazo de páginas, FIFO, LRU, anomalía de Belady, localidad de referencia, C++17.

2. Introducción

2.1 Contexto del problema

La memoria virtual permite que cada proceso disponga de un espacio de direcciones propio, mayor incluso que la memoria física disponible. El mecanismo que lo hace posible es la paginación: el espacio virtual se divide en páginas de tamaño fijo y la memoria física en marcos del mismo tamaño, y una tabla de páginas relaciona unas con otros. Cuando un proceso referencia una página que no reside en ningún marco se produce un fallo de página, y el sistema operativo debe traerla desde disco.

El problema aparece cuando no queda ningún marco libre: hay que elegir una página residente para desalojarla. Esa decisión es la política de reemplazo, y su calidad determina directamente el rendimiento del sistema, porque un fallo de página cuesta varios órdenes de magnitud más que un acceso a memoria. Una política

que desaloje sistemáticamente páginas que se van a volver a necesitar puede llevar al sistema al thrashing, un estado en el que se dedica más tiempo a intercambiar páginas que a ejecutar trabajo útil.

El objetivo de este trabajo no es únicamente implementar dos políticas clásicas, sino medir bajo qué condiciones cada una resulta preferible y cuánto margen de mejora queda respecto del máximo teórico alcanzable.

2.2 Objetivo general

Implementar un simulador de reemplazo de páginas que compare FIFO y LRU sobre las mismas cadenas de referencia, con estructuras de datos justificadas y eficiencia acreditada, y evaluar experimentalmente cómo influyen el tamaño de la memoria física y el patrón de acceso del programa sobre el número de fallos de página.

2.3 Objetivos específicos

- Leer cadenas de referencias desde archivo, con tolerancia a entradas malformadas y sin necesidad de recompilar para cambiar de cadena o de tamaño de marco.
- Implementar FIFO y LRU con estructuras de datos cuya complejidad se justifique explícitamente frente a las alternativas descartadas.
- Contabilizar fallos de página y aciertos, junto con la tasa de fallos, para cada algoritmo y configuración.
- Producir una traza paso a paso del estado de los marcos que permita seguir la simulación referencia por referencia.
- Verificar la corrección de las implementaciones contra valores publicados en la literatura, de modo que la validación no dependa del criterio del propio autor.
- Construir cadenas sintéticas con patrones de acceso contrastantes para determinar bajo qué condiciones cada algoritmo resulta superior.
- Reproducir y detectar automáticamente la anomalía de Belady, verificando que solo afecta a FIFO.
- Medir el tiempo de simulación sobre cadenas largas y relacionarlo con la complejidad algorítmica de cada implementación.

2.4 Alcance y limitaciones

El simulador modela exclusivamente la política de reemplazo sobre una secuencia de referencias a páginas ya dada. No se modela la traducción de direcciones, ni la tabla de páginas como

estructura, ni el coste real de la transferencia desde disco: el número de fallos es la métrica de calidad, y no se le asigna un tiempo de servicio. Tampoco se modela la memoria compartida entre procesos ni la distinción entre reemplazo local y global, dado que se simula un único proceso.

Queda fuera del alcance la segmentación, que constituye una forma alternativa de organizar el espacio de direcciones mediante bloques de tamaño variable. El reemplazo de páginas presupone bloques de tamaño fijo, de modo que ambos temas no son combinables en el mismo modelo; la comparación entre ambos enfoques se aborda en el marco teórico.

Se incluye deliberadamente un tercer algoritmo, el óptimo, que el enunciado no solicita. Su función es exclusivamente servir de cota inferior de referencia y se declara explícitamente como no implementable en un sistema real.

3. Marco Teórico

3.1 Paginación y fallo de página

La paginación divide el espacio de direcciones virtual de un proceso en bloques de tamaño fijo denominados páginas, y la memoria física en bloques del mismo tamaño denominados marcos. Página y marco son la misma unidad vista desde cada lado de la traducción [1]. La tabla de páginas mantiene la correspondencia entre ambas.

Su ventaja frente a la asignación contigua es que las páginas de un proceso pueden ubicarse en marcos no consecutivos, lo que elimina la fragmentación externa: cualquier marco libre sirve para cualquier página. A cambio aparece fragmentación interna, limitada al desperdicio dentro de la última página de cada proceso.

Cuando el proceso referencia una página que no reside en ningún marco se produce un fallo de página. Si existen marcos libres, la página se carga en uno de ellos; si no, es necesario elegir una página residente y desalojarla. Los fallos que ocurren la primera vez que se accede a cada página son inevitables y se denominan fallos obligatorios: constituyen un suelo que ninguna política de reemplazo puede cruzar.

3.2 FIFO

FIFO desaloja la página que lleva más tiempo cargada en memoria, con independencia de su uso reciente. Su atractivo es la simplicidad: basta una cola. Su debilidad es que la antigüedad de carga no guarda relación con la probabilidad de uso futuro, de modo que puede desalojar una página muy utilizada solo por haber entrado temprano.

3.3 LRU

LRU desaloja la página que lleva más tiempo sin ser referenciada. Se apoya en el principio de localidad de referencia: los programas tienden a reutilizar en el futuro próximo las páginas que han utilizado recientemente [1], [3]. Es una aproximación al comportamiento óptimo que emplea el pasado reciente como predictor del futuro inmediato, lo cual resulta razonable precisamente en la medida en que la carga exhiba localidad.

3.4 El algoritmo óptimo y la anomalía de Belady

El algoritmo óptimo desaloja la página cuyo próximo uso se encuentra más lejano en el futuro. Belady demostró que ninguna política puede producir menos fallos sobre la misma cadena, de modo que constituye una cota inferior. Requiere conocer la totalidad de la secuencia de referencias por adelantado, por lo que no es implementable en un sistema real; su valor es exclusivamente comparativo [1].

Belady describió además un fenómeno contraintuitivo que lleva su nombre: en FIFO, aumentar el número de marcos disponibles puede aumentar el número de fallos de página. La explicación reside en que FIFO no pertenece a la familia de los algoritmos de pila, aquellos en los que el conjunto de páginas residentes con n marcos está siempre contenido en el conjunto residente con $n+1$ marcos. LRU sí cumple esa propiedad de inclusión, y por ello está demostrado que no puede exhibir la anomalía.

3.5 Relación con la observación práctica en xv6

El kernel didáctico xv6 implementa la traducción de direcciones en vm.c, donde walkpgdir() recorre la tabla de páginas para localizar la entrada correspondiente a una dirección virtual y mappages() establece la correspondencia entre páginas y marcos. Sin embargo, xv6 no implementa ninguna política de reemplazo ni intercambio a disco.

La ausencia es deliberada y didácticamente reveladora: xv6 asume que la memoria física alcanza para todos los procesos activos, de modo que nunca necesita desalojar. Precisamente por eso el problema estudiado en este trabajo no aparece allí. Un gestor de memoria de producción requiere, además de la traducción que xv6 sí implementa, una política de reemplazo, un mecanismo de intercambio, bits de referencia y modificación mantenidos por hardware, y alguna forma de control del thrashing.

4. Diseño de la Solución

4.1 Arquitectura general

El proyecto separa el motor de simulación de las políticas de reemplazo. Un directorio común contiene la lectura y validación de la cadena de referencias, los tipos de resultado y el motor que recorre la secuencia; cada algoritmo reside en su propio encabezado y expone una interfaz idéntica, de modo que añadir una política nueva no obliga a modificar nada más.

El motor es una función plantilla y no una jerarquía con métodos virtuales. La razón es de medición: una llamada virtual por cada referencia, sobre cadenas de miles de accesos, introduciría una indirección en el camino de ejecución más frecuente y contaminaría el tiempo medido, que es una de las magnitudes comparadas.

4.2 FIFO: el puntero circular como cola implícita

La implementación mantiene el contenido de cada marco, un mapa de página a marco que responde en tiempo constante a la pregunta de si una página reside en memoria, y un puntero al marco que corresponde reemplazar.

Ese puntero hace innecesaria una cola explícita. Como los marcos se ocupan en orden y cada reemplazo avanza el puntero una posición de forma circular, el marco apuntado contiene siempre la página cargada hace más tiempo. El resultado es FIFO exacto en tiempo constante y sin memoria adicional respecto de la estrictamente necesaria.

4.3 LRU: lista enlazada con mapa de iteradores

El requisito no funcional exige una estructura eficiente para LRU y su justificación explícita. La dificultad reside en que cada acierto obliga a promover una página a la posición de más recientemente usada, y esa operación debe ser barata porque los aciertos son el caso frecuente.

Tabla 1: Alternativas consideradas para LRU.

Estructura	Localizar	Promover	Veredicto
Vector ordenado por uso	$O(n)$	$O(n)$	descartada
Lista enlazada sola	$O(n)$	$O(1)$	descartada
Lista + mapa de iteradores	$O(1)$	$O(1)$	elegida

La combinación resuelve ambas mitades del problema. El mapa localiza el nodo en tiempo constante y la operación splice lo traslada al frente reenganchando punteros, sin copiar el elemento. Un detalle relevante es que splice no invalida el iterador, de modo que no es necesario actualizar el mapa tras una promoción; de lo contrario el coste amortizado dejaría de ser constante. Es la estructura de un caché LRU clásico.

4.4 Óptimo: colas de ocurrencias

Para cada página se precalcula la lista de índices en que aparece dentro de la cadena. En cada referencia se descartan del frente los índices ya consumidos, con lo que el frente representa siempre el próximo uso de esa página. Al desalojar se elige la página cuyo frente resulte más lejano, y una página sin ocurrencias futuras se considera de distancia infinita y se desaloja de inmediato.

Cada índice entra y sale una única vez en toda la simulación, de modo que el precálculo es lineal y el mantenimiento amortizado constante. El coste por fallo es proporcional al número de marcos, al recorrer las páginas residentes en busca de la más lejana.

4.5 Instrumentación y verificación

Cada referencia produce un registro con la página solicitada, si constituyó acierto o fallo, la página desalojada en su caso, el marco afectado y el estado completo de los marcos tras la operación. Ese registro alimenta la traza paso a paso y solo se genera cuando se solicita explícitamente, de modo que su coste no se suma al tiempo medido en las corridas de medición.

Al término de cada corrida se comprueba que la suma de aciertos y fallos iguale el número de referencias procesadas. Todo acceso es necesariamente una cosa o la otra, nunca ambas ni ninguna; si la identidad no se satisface el programa devuelve un código de salida distinto de cero en lugar de informar cifras silenciosamente erróneas.

5. Implementación

5.1 Lenguaje y estructura

El proyecto está escrito en C++17 y utiliza exclusivamente la biblioteca estándar, sin dependencias externas. Compila con un único comando y no produce advertencias bajo -Wall -Wextra.

Tabla 2: Organización del código fuente.

Ubicación	Contenido
common/Cadena.h	Lectura y validación de la cadena
common/Simulacion.h	Tipos, métricas y motor de simulación
src/FIFO.h	Reemplazo por antigüedad de carga
src/LRU.h	Reemplazo por antigüedad de uso
src/Optimo.h	Cota inferior teórica (extra)
src/main.cpp	Línea de comandos, traza y salida
herramientas/generar.cpp	Generador de cadenas sintéticas
herramientas/reporte.cpp	Tablas y gráficos desde los CSV

5.2 Formato de entrada y tolerancia a errores

La cadena se lee desde archivo como números separados por espacios, comas o saltos de línea, admitiendo comentarios introducidos por almohadilla. Esto permite que cada archivo documente su propio origen y, en el caso de las cadenas sintéticas, incluya el comando exacto que las regenera.

Una entrada inválida no aborta la lectura: se informa por la salida de error indicando el número de línea y se descarta. Un archivo parcialmente corrupto sigue siendo utilizable y el usuario conoce exactamente qué se ignoró, en lugar de recibir un fallo total o, peor, un resultado silenciosamente incompleto.

5.3 Compilación y ejecución

```
make                # simulador y herramientas
make validar        # contrasta con la literatura
make traza          # traza paso a paso
make belady         # anomalía de Belady
make experimentos   # protocolo completo

./simulador --cadena data/corta.txt \
--marcos 3 --traza
```

Ningún parámetro requiere recompilar: el archivo de cadena, el número de marcos y el algoritmo se seleccionan por línea de comandos.

6. Resultados Experimentales

6.1 Entorno y protocolo

Tabla 3: Entorno de ejecución.

Elemento	Valor
Sistema	Ubuntu 26.04 LTS sobre VirtualBox
Compilador	g++ 15.2.0
Núcleos lógicos	6
Banderas	-std=c++17 -Wall -Wextra -O2

Se evaluaron cinco cadenas de referencia y hasta seis tamaños de marco por cadena, con los tres algoritmos en cada combinación, para un total de setenta y dos corridas. Las tres cadenas largas comprenden mil doscientas referencias sobre veinte páginas distintas y se generan con semilla fija, de modo que son reproducibles; además quedan versionadas en el repositorio, con lo que la reproducibilidad no depende de confiar en el generador.

6.2 Validación contra la literatura

La cadena 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 con tres marcos constituye el ejemplo canónico de la literatura sobre reemplazo de páginas [1], y es la misma que se empleó en clase. Sus valores están publicados, de modo que reproducirlos constituye una verificación externa: la corrección no depende del criterio del autor sobre su propio código.

Tabla 4: Validación sobre la cadena canónica con 3 marcos.

Algoritmo	Publicado	Obtenido	Estado
FIFO	15	15	coincide
LRU	12	12	coincide
ÓPTIMO	9	9	coincide

La comprobación está automatizada y puede ejecutarse en cualquier momento, incluida una eventual solicitud de reproducción en vivo.

6.3 Traza paso a paso

La traza reproduce la presentación habitual de la literatura: una fila por referencia y una columna por marco. El siguiente extracto compara ambos algoritmos en el punto exacto en que sus decisiones divergen, sobre la cadena canónica con tres marcos.

FIFO					LRU				
#	Ref	M0	M1	M2	#	Ref	M0	M1	M2
5	0	2	0	1	5	0	2	0	1
6	3	2	3	1	6	3	2	0	3
7	0	2	3	0	7	0	2	0	3

FIFO paso 6: desaloja 0 LRU paso 6: desaloja 1
FIFO paso 7: FALLO LRU paso 7: acierto

En el paso 6 ambos algoritmos deben desalojar una página. FIFO elige la página 0 por ser la cargada hace más tiempo, pese a que acababa de utilizarse en el paso 5. LRU elige la página 1, que no se referencia desde el paso 3. La consecuencia se observa de inmediato: cuando la página 0 vuelve a solicitarse en el paso 7, FIFO incurre en un fallo y LRU obtiene un acierto.

Ese único paso condensa toda la diferencia entre ambas políticas: la antigüedad de carga es un mal predictor del uso futuro, mientras que la antigüedad de uso resulta informativa siempre que la carga presente localidad.

6.4 Fallos frente al tamaño de marco

Las tablas siguientes recogen el número de fallos de cada algoritmo para cada tamaño de marco evaluado, con indicación del mejor en cada fila.

Tabla 5: Fallos de página — cadena canónica, 20 referencias.

Marcos	FIFO	LRU	ÓPTIMO	Mejor
3	15	12	9	OPTIMO
4	10	8	8	LRU
5	9	7	7	LRU

Tabla 6: Fallos de página — cadena con localidad, 1 200 referencias.

Marcos	FIFO	LRU	ÓPTIMO	Mejor
3	336	316	195	OPTIMO
4	108	71	59	OPTIMO
5	63	59	56	OPTIMO
8	59	59	47	OPTIMO
12	59	59	36	OPTIMO
16	59	59	28	OPTIMO

Tabla 7: Fallos de página — barrido secuencial, 1 200 referencias.

Marcos	FIFO	LRU	ÓPTIMO	Mejor
3	1200	1200	1074	OPTIMO
4	1200	1200	1012	OPTIMO
5	1200	1200	950	OPTIMO
8	1200	1200	764	OPTIMO
12	1200	1200	516	OPTIMO
16	1200	1200	268	OPTIMO

Tabla 8: Fallos de página — acceso aleatorio, 1 200 referencias.

Marcos	FIFO	LRU	ÓPTIMO	Mejor
3	1028	1030	777	OPTIMO
4	976	972	672	OPTIMO
5	904	914	587	OPTIMO
8	730	731	394	OPTIMO
12	486	481	217	OPTIMO
16	249	239	98	OPTIMO

6.5 El patrón de acceso decide qué algoritmo conviene

La comparación directa entre los tres patrones, a igualdad de marcos, permite contrastar la afirmación de que LRU aventaja a FIFO únicamente en presencia de localidad.

Tabla 9: Fallos con 5 marcos según el patrón de acceso, sobre 1 200 referencias.

Patrón	FIFO	LRU	ÓPT.	Resultado
Localidad	63	59	56	gana LRU
Secuencial	1200	1200	950	empatan
Aleatorio	904	914	587	gana FIFO

La hipótesis queda confirmada en los tres escenarios. Con localidad LRU obtiene menos fallos que FIFO y se sitúa a escasa distancia de la cota óptima. Sin localidad la ventaja desaparece: en el caso aleatorio FIFO resulta incluso marginalmente mejor, diferencia que no cabe interpretar como superioridad sino como ausencia de información aprovechable para cualquiera de las dos heurísticas.

El caso secuencial merece atención aparte. Ambos algoritmos fallan en la totalidad de las mil doscientas referencias, y siguen haciéndolo con dieciséis marcos. La causa es estructural: la cadena recorre veinte páginas en orden y vuelve a empezar, de modo que cuando el barrido regresa a la primera página esta ya fue desalojada para alojar a las restantes. Ninguna política basada en el historial de accesos puede acertar en ese escenario, y por ello el algoritmo óptimo sí mejora: no extrapola el pasado, consulta el futuro.

6.6 Anomalía de Belady

Se construyó una cadena específica para provocar la anomalía. Los resultados confirman su existencia sobre FIFO y su ausencia sobre LRU.

Tabla 10: Fallos sobre la cadena de Belady al aumentar los marcos.

Algoritmo	3 marcos	4 marcos	Comportamiento
FIFO	9	10	anomalía
LRU	10	8	normal
ÓPTIMO	7	6	normal

FIFO empeora al disponer de más memoria, que es precisamente el enunciado de la anomalía. El generador de reportes la detecta de forma automática recorriendo los resultados y señalando toda combinación en que un incremento de marcos produzca un incremento de fallos. Sobre las treinta combinaciones de cadena

y tamaño evaluadas, la marcó exclusivamente en FIFO, lo que constituye una verificación empírica de que LRU pertenece a la familia de algoritmos de pila.

6.7 Tiempo de simulación

Tabla 11: Tiempo de simulación en microsegundos, cadenas de 1 200 referencias con 5 marcos.

Cadena	FIFO	LRU	ÓPTIMO
Localidad	40.6	56.0	85.8
Secuencial	66.7	123.4	130.5
Aleatorio	89.0	150.7	145.8

Las cifras son coherentes con la complejidad de cada implementación. FIFO resulta el más rápido: su operación por referencia se reduce a una consulta en tabla y, en caso de fallo, a una escritura y un incremento. LRU añade la manipulación de la lista de uso en cada acceso, incluidos los aciertos. El algoritmo óptimo paga el recorrido de las páginas residentes en cada fallo, además del precálculo inicial de las ocurrencias.

Debe subrayarse que estas magnitudes miden el coste del algoritmo dentro del simulador y no guardan relación con el coste real de un fallo de página en un sistema operativo, dominado por la transferencia desde disco. En un sistema real la diferencia de microsegundos entre políticas resulta irrelevante frente a un solo fallo evitado.

7. Evaluación ISO/IEC 25010

Autoevaluación conforme a la rúbrica institucional, en escala de Excelente (4) a Insuficiente (1), justificada con evidencia propia del proyecto.

Tabla 12: Autoevaluación ISO/IEC 25010.

Característica	C.	Justificación
Rendimiento	4	Se mide el tiempo de simulación con reloj monótono y se reporta por cada combinación de cadena, tamaño de marco y algoritmo (Tabla 11). Las cifras se relacionan explícitamente con la complejidad de cada estructura de datos: constante por referencia en FIFO y LRU, proporcional a los marcos por fallo en el óptimo. Las estructuras se eligieron comparando alternativas con su coste (Tabla 1).
Fiabilidad	4	Las tres implementaciones reproducen los valores publicados en la literatura para la cadena canónica (Tabla 4), verificación externa automatizada y ejecutable en vivo. Cada corrida comprueba además que aciertos más fallos igualen las referencias, devolviendo código de salida distinto de cero si la identidad falla. La lectura de entrada tolera líneas malformadas informando su ubicación.
Usabilidad	3	La interfaz ofrece ayuda en línea, mensajes de error específicos por argumento, y una traza que reproduce la presentación de los libros con resaltado por color del resultado de cada referencia. No alcanza el máximo porque la interpretación de la traza requiere conocer la convención de columnas: la salida no explica por sí misma qué representa cada marco sin consultar la documentación.
Compatibilidad	3	El código emplea únicamente la biblioteca estándar de C++17, sin llamadas dependientes de plataforma, y el generador de reportes ofrece un modo cuya salida se limita al rango ASCII. La calificación no alcanza el máximo porque, a diferencia del proyecto anterior, la compilación en un segundo entorno no se ha verificado todavía de forma directa; se declara como limitación en lugar de afirmarse por ausencia de dependencias.
Mantenibilidad	4	Cada política reside en su propio encabezado con interfaz común, de modo que incorporar una nueva no exige modificar el motor ni la instrumentación, como demuestra la inclusión del algoritmo óptimo. Las decisiones de diseño están documentadas en el propio código, incluida la justificación de las estructuras descartadas. El archivo de compilación declara los encabezados como dependencia.

Las dos calificaciones inferiores al máximo corresponden a limitaciones reales y verificables. Se ha evitado afirmar

cualquier propiedad que no cuente con un comando capaz de reproducirla.

8. Discusión de Resultados

8.1 Qué algoritmo resultó superior y bajo qué condiciones

La respuesta no es única y esa es la conclusión más útil del trabajo. LRU resultó superior únicamente cuando la carga presentaba localidad de referencia; sin ella la ventaja desapareció por completo, e incluso FIFO obtuvo un resultado marginalmente mejor en el caso aleatorio.

La explicación es que LRU no es intrínsecamente mejor: es una heurística que apuesta a que el pasado reciente predice el futuro inmediato. Cuando esa apuesta es acertada, como ocurre en programas reales con bucles y estructuras que se recorren varias veces, LRU acierta. Cuando la carga carece de localidad, la información que LRU mantiene y sobre la que decide simplemente no es predictiva, y el coste adicional de mantenerla no se traduce en beneficio alguno.

Resulta ilustrativo que en la cadena con localidad, a partir de ocho marcos, tanto FIFO como LRU se estabilicen en el mismo número de fallos sin mejorar por más memoria que se les asigne. Ese valor corresponde a los fallos obligatorios: las páginas deben cargarse al menos una vez y ninguna política de reemplazo puede evitarlo. El algoritmo óptimo continúa descendiendo por debajo de esa cifra únicamente porque, con memoria suficiente, evita recargas que las otras dos sí realizan antes de alcanzar la saturación.

8.2 Relación con la teoría

El paso 6 de la traza explica toda la diferencia. La decisión divergente documentada en la Sección 6.3 no es una anécdota sino el mecanismo completo en miniatura: FIFO desaloja una página recién utilizada porque solo consulta la antigüedad de carga, y falla en la referencia siguiente. La diferencia agregada de tres fallos sobre la cadena canónica es la acumulación de decisiones de esa naturaleza.

La anomalía de Belady se comportó exactamente como predice la teoría. Se manifestó en FIFO y en ninguna de las treinta combinaciones evaluadas apareció en LRU. Esto constituye una verificación empírica de la propiedad de inclusión de los algoritmos de pila: si una página reside en memoria con n marcos, también reside con n+1, de modo que ampliar la memoria no puede introducir fallos nuevos. FIFO no satisface esa propiedad porque su criterio de desalojo es independiente del uso.

El barrido secuencial delimita lo que ninguna heurística puede lograr. Una tasa de fallos del cien por cien con dieciséis marcos sobre veinte páginas demuestra que el problema no reside en la política elegida sino en la relación entre el conjunto de trabajo y la memoria disponible. Es la manifestación en pequeño del fenómeno que en un sistema real conduce al thrashing, y explica por qué la solución práctica ante ese síntoma no consiste en cambiar de algoritmo de reemplazo sino en reducir el grado de multiprogramación o ampliar la memoria física.

8.3 Limitaciones

El número de fallos no se traduce a tiempo real. El simulador cuenta fallos pero no les asigna coste. En un sistema real un fallo implica una transferencia desde disco cuyo tiempo domina cualquier otra consideración, de modo que las diferencias de microsegundos entre algoritmos carecen de significado práctico frente a un solo fallo evitado.

Las cadenas sintéticas son modelos, no trazas reales. El patrón con localidad se genera mediante un conjunto de trabajo que se desplaza a intervalos regulares, lo cual es una idealización. Una traza capturada de un programa real presentaría transiciones menos limpias y probablemente una mezcla de los tres patrones evaluados por separado.

No se modela el bit de referencia ni sus aproximaciones. LRU exacto resulta costoso de implementar en hardware, razón por la cual los sistemas reales emplean aproximaciones como el algoritmo de segunda oportunidad o el reloj. Compararlas habría permitido evaluar cuánta precisión se sacrifica a cambio de viabilidad, pero excede el alcance solicitado.

Se simula un único proceso. No se modela la competencia por marcos entre procesos ni la distinción entre reemplazo local y global, que es donde el thrashing aparece con toda su gravedad en un sistema real.

La compatibilidad no se verificó en un segundo entorno. A diferencia del proyecto anterior, la compilación en Windows no se ha comprobado de forma directa. Se declara como limitación y no se afirma por ausencia de dependencias de plataforma.

9. Conclusiones

El proyecto produjo un simulador de reemplazo de páginas con tres políticas implementadas sobre una interfaz común, cuya corrección se acredita reproduciendo valores publicados en la literatura en lugar de descansar en el criterio del propio autor. Las estructuras de datos se eligieron comparando explícitamente alternativas y su coste, y la elección para LRU consigue tiempo constante tanto en la localización como en la promoción de páginas.

El resultado central es que no existe una política universalmente superior. LRU aventaja a FIFO exclusivamente cuando la carga presenta localidad de referencia; sin ella la ventaja se anula. Esto reformula la pregunta habitual: no se trata de cuál algoritmo es mejor, sino de qué supuestos sobre la carga justifican cada heurística.

Tres aprendizajes se derivan del desarrollo. El primero es que una sola decisión de reemplazo condensa toda la diferencia conceptual entre ambas políticas: en el paso seis de la cadena canónica, FIFO desaloja una página utilizada un instante antes por el solo hecho de haber sido cargada primero, y falla en la referencia siguiente. Comprender ese paso equivale a comprender el algoritmo.

El segundo es que el comportamiento contraintuitivo de la anomalía de Belady no es una curiosidad histórica sino una consecuencia estructural verificable: se reprodujo en FIFO y estuvo ausente en LRU en la totalidad de las combinaciones

evaluadas, lo que confirma empíricamente la propiedad de inclusión de los algoritmos de pila.

El tercero es que existen escenarios donde la política de reemplazo es irrelevante. El barrido secuencial produjo una tasa de fallos del cien por cien con cualquier algoritmo y cualquier tamaño de memoria evaluado, lo que demuestra que cuando el conjunto de trabajo excede la memoria disponible el problema deja de ser algorítmico. Incorporar el algoritmo óptimo como cota inferior fue lo que permitió distinguir esa situación de una simple mala elección de política.

Como extensiones fuera del alcance se identifican: implementar las aproximaciones de segunda oportunidad y reloj para cuantificar cuánta precisión se sacrifica frente a LRU exacto; modelar varios procesos compitiendo por marcos, para observar el thrashing en las condiciones en que realmente se produce; y sustituir las cadenas sintéticas por trazas capturadas de programas reales.

10. Referencias

- [1] A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2018.
- [2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.^a ed. Boston, MA, EE. UU.: Pearson, 2015.
- [3] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Madison, WI, EE. UU.: Arpaci-Dusseau Books, 2018.
- [4] ISO/IEC, *ISO/IEC 25010:2011 — Systems and software engineering — SQuaRE — System and software quality models*. Ginebra, Suiza: ISO, 2011.